TECHNICAL REFERENCE

Power Glove Vision Configuration Reference

Active files, installed copies, fields, secrets, and generated state.

2026 EDITION / IAIN BENNETT / MIT LICENSE

This guide is for anyone installing or adapting Power Glove Vision on their own Arduino UNO Q and RetroPie system. It explains which settings are safe to change, where the active copies live, how game profiles are selected, and how to recover from a bad configuration.

Use the installation guide for the initial deployment and pairing procedure. Return here when you need to change a host, camera, game, gesture threshold, or network setting.

> **KEEP THE TOKEN PRIVATE** Power Glove Vision uses one shared token to authenticate controller packets and profile changes. Never paste it into an issue, screenshot, command line, public backup, or Git commit.

The three places configuration lives

Power Glove Vision runs on two computers. A repository template is not always the file the running system reads.

Location	What it controls	Preferred way to change it
UNO Q Setup page	Console address, controller port, startup profile, camera, and pairing token	Browser Setup page
UNO Q application files	Gesture sensitivity and advanced runtime defaults	Edit only when tuning is required
RetroPie `/etc/powerglove/`	UNO Q address, per-game profile selection, and receiver token	Protected files on RetroPie

The examples under the repository's `config/` directory are installation templates. Editing them does not change an already installed system. The active copies are identified in each section below.

In commands and examples, replace these placeholders:

Placeholder	Replace with
`UNO-Q-NAME.local`	Your UNO Q hostname or reserved IP address
`RETROPIE-NAME.local`	Your RetroPie hostname or reserved IP address
`/home/arduino/ArduinoApps/powerglove-vision`	Your UNO Q App Lab application directory, if different

Recommended setup workflow

1. Configure the UNO Q from its Setup page.
2. Pair the UNO Q with RetroPie so both machines receive the same private token.
3. Confirm the RetroPie launcher points to the UNO Q.
4. Add your exact ROM filenames to the game registry.

5 Install the RetroArch autoconfiguration and launch a registered game.

6 Tune gesture thresholds only after the standard profiles work.

UNO Q settings

Open the ordinary Setup page at:

```
http://UNO-Q-NAME.local:8088/setup
```

Pairing uses the secure Setup page instead:

```
https://UNO-Q-NAME.local:8443/setup
```

The secure page uses a locally generated certificate. During pairing, compare the certificate fingerprint shown by the browser with the identifier shown on the UNO Q matrix before entering the one-time PIN.

Settings shown in the browser

Setting	Default	Meaning and recommendation
Console hostname or IP	Empty (not configured)	Set your RetroPie hostname (`RETROPIE-NAME.local` in examples) or a reserved LAN address and pair through Connection before starting controls. Learn and local settings work without a destination. Existing saved destinations are preserved.
Controller port	`55355`	UNO Q to RetroPie controller-state port. Leave it at the default unless both ends are changed.
Startup profile	`bad_street_brawler`	Profile used before a registered game selects another one.
Tracking aid	`none`	`none`, `white`, or `black`. In the current release this is an informational diagnostic label; it does not change MediaPipe tracking.
Camera	`auto`	Prefer `auto`. Use a number from `0` through `99` only when automatic selection chooses the wrong capture device.
Generate a new token	Off	Rotates the shared secret. This immediately breaks the existing pairing until RetroPie is paired again.

Selecting **Save** validates the fields, writes them atomically with private permissions, and restarts the vision worker. Hold a neutral hand in view and center it again after the restart.

The Dashboard profile selector changes only the current active profile. It does not rewrite `device.json` or change the Setup page's startup profile. RetroPie may replace a Dashboard selection when a game starts or ends.

The first vision activation opens the camera and loads the gesture tracker, so it can take longer than switching between active profiles. Dashboard and Learn show **Starting camera and gesture tracking** with elapsed time until vision is active. Centering is disabled during initialization. The camera is not pre-warmed at boot when **Gestures off** is selected.

The Learn page uses a temporary practice session. Entering Learn starts camera capture and recognition even if the selected profile is **Gestures off**; the practice engine uses Program H internally without changing the selected profile. Controller packets remain suppressed. Leaving Learn restores the selected profile and its camera state. A Dashboard load or refresh clears any stale practice session, and a six-second lease timeout provides the same recovery if a browser

closes without sending its normal release request. An existing Learn tab cannot reactivate practice after a Dashboard reset; reload Learn to begin a new practice session.

More than one Learn tab may be open. Vision remains active until the last tab closes or its lease expires.

The supplied profiles activate curl at 0.50 and release below 0.35. Learn uses the same held finger state as gameplay, so small changes during a hold do not reset the lesson. Bending the middle knuckle alone can qualify; bending the base knuckle is not required. Menu poses have separate thresholds. V/Start requires index and middle curl below 0.28, with ring and pinky above 0.42, held for about 0.7 seconds. Thumbs-up/Select requires thumb curl below 0.32 and all four fingers above 0.42, with the same deliberate hold.

Camera finger curl uses the strongest joint bend rather than averaging bends, including the base knuckle for the four fingers. The thumb uses its two outer joints. **Live hand measurements** in Learn displays exact curl values, the action threshold, magnified landmarks, and forward movement relative to the centered hand size. Push recognition stays active while forward movement exceeds its hysteresis threshold; Learn does not rely on a one-frame event.

Camera finger curl uses MediaPipe 3D world landmarks when available. If those are absent, normalized depth is used with image aspect-ratio correction. Palm movement remains based on image coordinates. The legacy Arduino landmark bridge retains its existing 2D interpretation because its depth units are not assumed to match MediaPipe. Gesture thresholds are unchanged.

Learn teaches A (index curl), B (thumb curl), and GLOVE ZAP (forward push), with live practice indicators. Game-specific mappings remain separate. Learn awards a Glove Master achievement once all eleven lessons are completed; skipped lessons must be revisited. Start again clears session progress. The Calibrate button turns red while calibration is active, then blue with a brief completion confirmation. Browser preview encoding is capped at 15 fps; controller status continues at inference rate. Status fields `inference_ms` and `send_ms` measure processing and local send time, not full camera-to-game latency. Centering happens when a new vision engine starts. Learn uses the general practice profile; changing into or out of a different profile starts a fresh engine and centers again. Use Calibrate with a relaxed hand if needed.

Each Learn lesson shows a gesture illustration. Start uses a V sign; Select uses a thumbs-up with the other four fingers closed. Both must remain steady for about 0.7 seconds. The lesson accepts the confirmed pose even after its short controller pulse ends. Learn shows precise curl values from 0 to 1; the Dashboard's compact finger display uses 0 to 3. If directions appear instead, keep the palm near center and check the finger readings; direction output alone does not establish a model bias.

Start controller and **Stop controller** change live output only. The controller deliberately starts stopped after an application or system restart; this prevents hand motion from navigating menus before you are ready.

Active UNO Q device file

The Setup page maintains this private file inside the application:

```
/home/arduino/ArduinoApps/powerglove-vision/data/device.json
```

A typical active file has this shape:

```
JSON
{
  "receiver": "RETROPIE-NAME.local",
  "port": 55355,
  "token": "private-random-value-created-by-the-application",
  "profile": "bad_street_brawler",
  "glove_color": "none",
  "camera": "auto"
}
```

Use the Setup page for routine changes. If you must edit the JSON directly, stop the application first, keep the token unchanged, validate the file, and restore mode `0600`. JSON does not allow comments or trailing commas.

```
SH
python3 -m json.tool data/device.json >/dev/null
chmod 0600 data/device.json
```

Deleting `device.json` causes Power Glove Vision to create a new token and first-run defaults. You must then pair RetroPie again.

Camera selection

`auto` searches stable `/dev/v4l/by-id/` capture-device links first, ignores known codec-only video nodes, and then considers ordinary `/dev/video*` capture devices. This is the most reliable choice when USB enumeration changes after a reboot.

Use an explicit number only for troubleshooting. If `0` selects `/dev/video0`, for example, that number may refer to different hardware after devices are reconnected. Keep the camera on a powered USB hub when the UNO Q cannot supply stable power by itself.

Supported startup profiles

The valid profile identifiers are:

```
bad_street_brawler
super_glove_ball
program_a  program_b  program_c  program_d  program_e
program_f  program_g  program_h  program_i
```

The startup profile does not assign a profile to a ROM. Per-game selection is controlled by the RetroPie game registry described below.

Pairing and token management

Both machines must hold the same token:

Machine	Active private file
UNO Q	Application `data/device.json`, in the `token` field
RetroPie	`/etc/powerglove/token`

Use one-time-code pairing whenever possible:

```
SH
sudo /opt/powerglove/bin/powerglove-pair
```

Leave that command running on RetroPie, then complete pairing at `https://UNO-Q-NAME.local:8443/setup`. The code is single use and expires after two minutes. Password pairing is also available when RetroPie accepts SSH password login; the password is used for one encrypted operation and is not stored.

The RetroPie token must contain at least 16 characters and should remain owned by `root`, readable by the `input` group, and inaccessible to other users:

```
SH
sudo chown root:input /etc/powerglove/token
sudo chmod 0640 /etc/powerglove/token
```

If you rotate the token from Setup, pair again immediately. Do not try to make the new value match by passing it as a command-line argument; process listings and shell history can expose it.

RetroPie connection settings

The active launcher file is:

```
/etc/powerglove/launcher.json
```

It tells the runcommand hooks where to send profile changes when a game starts or exits.

```
JSON
{
  "uno_q": "UNO-Q-NAME.local",
  "port": 55356,
  "token_file": "/etc/powerglove/token",
  "registry": "/etc/powerglove/games.json",
  "timeout": 0.4
}
```

Field	Meaning
`uno_q`	UNO Q hostname or reserved address reachable from RetroPie.
`port`	RetroPie to UNO Q profile-control port. This is `55356`, not the controller-state port.
`token_file`	Protected shared-token file. Keep the token out of this JSON file.
`registry`	Active ROM-to-profile mapping.
`timeout`	Seconds to wait for each acknowledgement. The hook retries up to three times and never prevents a game from launching.

Validate changes before launching a game:

```
SH
python3 -m json.tool /etc/powerglove/launcher.json >/dev/null
```

Use a `.local` hostname when multicast DNS is reliable on your LAN. A reserved DHCP address is a useful fallback. Do not use an address that may later be assigned to another device.

Register games and select profiles

The active game registry is:

```
/etc/powerglove/games.json
```

Power Glove Vision matches the exact ROM basename, including its extension, without regard to letter case. Directory names are ignored. Automatic profile selection currently applies only when RetroPie reports the system as `nes` or `famicom`; other systems turn gesture control off.

```
JSON
{
  "games": {
    "Bad Street Brawler (USA).nes": "bad_street_brawler",
    "Super Glove Ball (USA).zip": "super_glove_ball",
    "Joust (USA).nes": "program_b"
  }
}
```

Add the exact filename shown in EmulationStation. Zipped and 7-Zip copies need their own entries because `.nes`, `.zip`, and `.7z` are different basenames.

Included game	Profile
Bad Street Brawler	`bad_street_brawler`
Super Glove Ball	`super_glove_ball`
Joust	`program_b`
Gyruss	`program_c`
Defender II	`program_e`
Sesame Street 1-2-3	`program_f`
Gun Smoke	`program_g`
Knight Rider	`program_i`

The table above is the complete set of games recognized automatically by the shipped registry. Programs A, D, and H are fully implemented profiles rather than omitted games: `program_a` is a pinball control scheme, `program_d` reverses all four directions for challenge or accessibility use, and `program_h` provides general-purpose movement with pulsed buttons. They deliberately have no default ROM assignment.

Any appropriate NES or Famicom ROM can use one of those profiles after you add its exact basename to the `games` object. Every profile value must be one of the supported identifiers; an unknown value invalidates the registry.

```
SH
sudo python3 -m json.tool /etc/powerglove/games.json >/dev/null
```

Test profile communication independently of a game:

```
SH
sudo /opt/powerglove/bin/powerglove-profile \
  --uno-q UNO-Q-NAME.local \
  --token-file /etc/powerglove/token \
  --profile program_b
```

Use `--profile off` to stop gesture output. A successful request prints an acknowledgement and starts the matrix glove attract animation. In this healthy idle state the camera and MediaPipe tracker are closed, while the website and authenticated profile listener remain available.

Tune gesture sensitivity

The UNO Q reads gesture thresholds from the application's active `config/profiles.json`:

```
/home/arduino/ArduinoApps/powerglove-vision/config/profiles.json
```

Make a private backup before editing it. The Setup page does not expose these advanced thresholds.

Threshold fields

Field	What it measures	Effect of lowering the value
`move_on`	Palm displacement from center, normalized by palm size	Movement activates sooner
`move_off`	Palm displacement at which active movement releases	Movement stays active farther back toward center
`curl_on`	Normalized finger curl, where `0` is straight and `1` is tightly curled	Curl actions activate with less bend
`curl_off`	Curl amount at which an active curl releases	Curl stays active until the finger is straighter
`roll_on`	Wrist rotation from the centered angle	Roll actions activate with less rotation
`roll_off`	Rotation at which active roll releases	Roll stays active closer to neutral
`push_on`	Relative increase in apparent hand size from center	Push actions activate with less forward movement
`push_off`	Depth change at which an active push releases	Push stays active closer to the centered depth
`pulse_hz`	Repetition rate for profiles that pulse an action	Repeated actions become slower
`loss_release_ms`	Tracking-loss delay before all controls release	Controls release sooner after the hand disappears

For each gesture, keep the `_off` value lower than its `_on` value. The gap is hysteresis: it prevents a value near the activation point from rapidly turning on and off. A very large gap can make the control feel sticky.

The supplied defaults are:

```JSON
{
  "move_on": 0.38,
  "move_off": 0.24,
  "curl_on": 0.50,
  "curl_off": 0.35,
  "roll_on": 0.58,
  "roll_off": 0.40,
  "push_on": 0.34,
  "push_off": 0.18,
  "pulse_hz": 7.0,
  "loss_release_ms": 120
}
```

`super_glove_ball` has a more responsive movement pair of `0.32` and `0.20`, a higher roll pair of `0.70` and `0.50`, an `8.0` Hz pulse rate, and slightly higher push thresholds. Profiles A through I use `program_defaults` unless an exact profile object such as `program_b` is added.

Change one pair at a time in steps of approximately `0.02` to `0.05`, then test from the same camera position. Useful adjustments include:

- Lower `move_on` if directional movement requires too much travel.
- Raise `move_off` if a direction remains held after returning toward center.
- Raise an `_on` value when an action triggers unintentionally.
- Increase `pulse_hz` when a repeating action is too slow.
- Keep `loss_release_ms` short enough to release safely but long enough to tolerate a few missed camera frames.

Validate the file, restart the UNO Q application from App Lab, and center the hand again. Gesture-to-button assignments are implemented by each profile in the application; threshold changes adjust sensitivity but do not remap buttons.

RetroPie receiver and virtual controller

The receiver verifies authenticated UDP packets and creates a Linux `uinput` gamepad named `PowerGlove Vision`. Its installed service is:

```
/etc/systemd/system/powerglove-receiver.service
```

The supplied service listens on all local interfaces at UDP port `55355`, reads `/etc/powerglove/token`, and releases all controls after 250 milliseconds without a valid packet. If you change the controller port in UNO Q Setup, add the same `--port` value to the service's `ExecStart`, then reload and restart:

```SH
sudo systemctl daemon-reload
sudo systemctl restart powerglove-receiver.service
```

The companion `powerglove-receiver.timer` starts the receiver 45 seconds after boot. This lets EmulationStation finish its initial controller scan first. Keep the timer enabled and the service itself disabled for boot activation. Starting the receiver too early can cause frontend pauses and conflicts with other USB devices, such as a BitPixel display.

```
SH
systemctl is-enabled powerglove-receiver.timer
systemctl is-enabled powerglove-receiver.service
sudo systemctl status powerglove-receiver.service
sudo journalctl -u powerglove-receiver.service -n 100 --no-pager
```

The virtual gamepad appears only after the first authenticated controller packet. On the UNO Q, select **Start controller** and show a centered hand before deciding that the device is missing.

RetroArch autoconfiguration

Install the supplied mapping at:

```
/opt/retropie/configs/all/retroarch/autoconfig/PowerGlove Vision.cfg
```

It matches the virtual device name plus vendor and product IDs `1:1`, uses the `udev` input driver, and maps D-pad directions, A, B, Start, Select, and four axes to a standard RetroPad. It does not configure an I-PAC, 8BitDo controller, or any other physical controller.

If RetroArch has a hand-written override for this device, remove or reconcile that override before diagnosing the supplied autoconfiguration.

UNO Q shutdown helper

The standard installation includes the host shutdown helper so Dashboard and Setup can halt Linux cleanly. It consists of two systemd units and one boot-time readiness rule:

Repository file	Installed path	Purpose
`uno-q/powerglove-system-shutdown.path`	`/etc/systemd/system/powerglove-system-shutdown.path`	Watches for one fixed shutdown request in the application's private data directory
`uno-q/powerglove-system-shutdown.service`	`/etc/systemd/system/powerglove-system-shutdown.service`	Removes that request and asks systemd to halt Linux cleanly
`uno-q/powerglove-system-shutdown.conf`	`/etc/tmpfiles.d/powerglove-system-shutdown.conf`	Recreates the unprivileged readiness marker at boot or after application replacement

Install them with `scripts/install-uno-q-shutdown-helper.sh`. The installer also creates the private `data/.shutdown-enabled` marker that allows the web UI to offer the action. The tmpfiles rule restores that marker at boot. The application cannot use this mechanism to execute an arbitrary privileged command; it can only create the fixed request after an explicit confirmation.

Do not change the request path in only one component. The web application, path unit, service, and marker must continue to agree. After installation, confirm that `powerglove-system-shutdown.path` is enabled and active.

Network ports and trust boundary

Keep Power Glove Vision on a trusted home or cabinet LAN. Do not forward these ports through a router or expose them directly to the Internet.

Port	Direction	Purpose
UDP `55355`	UNO Q to RetroPie	Authenticated live controller state
UDP `55356`	RetroPie to UNO Q	Authenticated game-profile requests and acknowledgements
TCP `8088`	Browser to UNO Q	Dashboard, Help, Learn, and ordinary Setup UI
TCP `8443`	Browser to UNO Q	TLS Setup and pairing workflow
TCP `55357`	UNO Q to RetroPie	Temporary one-time-code pairing helper

The two UDP ports serve different purposes despite their similar numbers. The `port` in UNO Q `device.json` is normally `55355`; the `port` in RetroPie `launcher.json` is normally `55356`.

The ordinary dashboard is reachable by other devices on the LAN. Pairing and shutdown require additional protections, but the project assumes that the LAN itself is trusted. Review SECURITY.md before using a shared, guest, school, or public network.

UNO Q matrix artwork

The built-in display is a 13-by-8 monochrome blue LED matrix with eight brightness levels. Treat it as a very small dot-matrix display in the spirit of a pinball DMD or monochrome BitPixel display, rather than as a miniature screen. The sketch encodes status, pairing, profile identifiers, and the gestures-idle Power Glove attract sequence.

Matrix artwork should use broad motion, recognizable silhouettes, and strong separation between the subject and its brightest effect. A moving spark needs a dim body underneath it, a medium halo, and a full-bright core. Pulses should emphasize an outline instead of illuminating every pixel at maximum brightness, which erases the shape on the physical display. Transitional objects need to cross several columns and remain visible for more than one frame; isolated one-pixel changes are easily lost to persistence and viewing angle.

Always judge animation timing and grayscale on the physical UNO Q. A source grid or browser mock-up is useful for finding malformed frames, but it cannot reproduce LED bloom, exposure, or perceived persistence. A short video covering several complete loops is the preferred review artifact for later refinements.

Files most users should not edit

File or directory	Purpose
`app.yaml`	Arduino App Lab application metadata and exposed ports
`sketch/sketch.yaml`	UNO Q sketch platform and pinned Arduino library dependencies
`pyproject.toml`	Python package metadata, supported interpreter range, and optional dependencies
`python/worker-wheels/`	Platform-specific MediaPipe worker dependency supplied by the App Lab installation ZIP
`data/models/hand_landmarker.task`	Checksum-verified model downloaded when vision is first activated
`data/uv-cache/` and `data/uv-python/`	Generated private worker runtime and package cache
`.cache/app-compose.yaml`	App Lab generated container configuration
`data/.shutdown-enabled`	Marker installed by the optional fixed-purpose shutdown helper
`output/pdf/`	Generated PDF editions; public editions are served by Help, while the cabinet quick reference remains private

Changing manifests can prevent App Lab from starting the application. Generated files and downloaded runtime assets should not be committed or added to an App Lab installation ZIP. The installation ZIP is built as:

```
output/app-lab/PowerGlove-Vision-Uno-Q.zip
```

The installation ZIP intentionally excludes private `data/`, downloaded models, caches, tests, Git metadata, and the cabinet-specific quick-reference PDF. It includes only the nine allowlisted public PDF editions used by Help. The pinned Google Hand Landmarker model downloads and passes a SHA-256 check when an active profile first needs vision. Gestures-idle mode does not open it.

Automated quality and package verification

Maintainers use `.github/workflows/quality.yml` for pull requests, pushes to `main`, and manual runs. It tests supported Python versions, checks Python and shell syntax, validates JSON, audits source and documentation, rebuilds and inspects the PDF set, and verifies the App Lab installation ZIP. The workflow has read-only repository permissions and uses no deployment credentials.

Ordinary installers do not need to edit this workflow. If you fork the project, keep live UNO Q credentials and cabinet deployment out of hosted CI; deployment should remain an explicit action from a trusted machine.

Back up and update safely

Back up custom configuration before replacing an installation:

Item	Why it matters
UNO Q `data/device.json`	Contains device settings and the private token
UNO Q `config/profiles.json`	Contains any custom sensitivity values

Item	Why it matters
RetroPie `/etc/powerglove/games.json`	Contains local ROM mappings
RetroPie `/etc/powerglove/launcher.json`	Contains local host and path settings
RetroPie `/etc/powerglove/token`	Contains the matching private token
RetroArch `PowerGlove Vision.cfg`	Contains any deliberate local mapping changes

Store token-bearing backups privately with restricted permissions. The supplied Wi-Fi deployment script preserves the UNO Q `data/` directory. The App Lab installation ZIP never contains your token or downloaded model.

After an update, confirm that the active files under `/etc/powerglove/` still contain your local hostnames and ROM names. Updating repository templates does not automatically migrate active configuration.

Troubleshooting by symptom

Symptom	Configuration checks
Dashboard works but no virtual controller appears	Start the controller, show a centered hand, verify the receiver service and shared token, then check UDP `55355`.
Controller appears but a game uses the wrong gestures	Confirm the system is `nes` or `famicom` and the exact ROM basename exists in `/etc/powerglove/games.json`.
Game launches slowly while UNO Q is offline	Confirm `timeout` remains near `0.4`; the hook retries but must never block game launch indefinitely.
Profile command is not acknowledged	Check the UNO Q name, UDP `55356`, pairing token, and the UNO Q application status.
Gestures off shows a blinking X	Update Power Glove Vision; Gestures off should show the glove attract animation and must not open the camera.
Camera disappears after reboot	Return Camera to `auto`, check powered-hub and cable stability, and inspect the UNO Q dashboard error.
Movement triggers too late	Center again first; if repeatable, lower `move_on` slightly for the active profile.
Direction remains stuck	Raise `move_off` slightly, keep it below `move_on`, and verify tracking-loss release.
Pairing suddenly fails after a Setup change	A rotated token invalidates the old pairing; run the pairing flow again.
EmulationStation pauses or another USB device behaves unexpectedly at boot	Verify receiver startup is controlled by the 45-second timer and the service is not independently enabled at boot.

Configuration file catalog

Repository file	Active or installed copy	Used by
`config/device.example.json`	UNO Q application `data/device.json`	Vision supervisor and Setup UI
`config/profiles.json`	UNO Q application `config/profiles.json`	Gesture engine
`config/games.json`	RetroPie `/etc/powerglove/games.json`	Launch hook and profile selector
`config/launcher.example.json`	RetroPie `/etc/powerglove/launcher.json`	RetroPie launch and exit hooks
`retropie/retroarch/PowerGlove Vision.cfg`	RetroArch autoconfig directory	RetroArch input system
`retropie/powerglove-receiver.service`	`/etc/systemd/system/`	Privileged virtual-controller receiver
`retropie/powerglove-receiver.timer`	`/etc/systemd/system/`	Delayed boot activation
`uno-q/powerglove-system-shutdown.path`	`/etc/systemd/system/`	Fixed shutdown request watcher
`uno-q/powerglove-system-shutdown.service`	`/etc/systemd/system/`	Fixed clean-shutdown action
`uno-q/powerglove-system-shutdown.conf`	`/etc/tmpfiles.d/`	Boot-time shutdown readiness marker
`.github/workflows/quality.yml`	GitHub Actions	Automated tests and release verification
`app.yaml`	UNO Q application root	App Lab
`sketch/sketch.yaml`	UNO Q application sketch directory	Arduino build system
`pyproject.toml`	Repository or deployed application root	Python packaging and tests

When a change goes wrong, restore the last known-good active file rather than copying every repository template over the installation. Validate JSON, restart only the affected service or application, and test packet delivery before changing controller mappings.

Local hostname resolution inside App Lab

The app resolves `.local` IPv4 names through the UNO Q host's Avahi socket. The deployment script mounts `/run/avahi-daemon` read-only into the container; this is a directory mount so daemon restarts can replace the socket safely. Answers are cached for five seconds, then refreshed, allowing DHCP changes. Gameplay, connection testing, and both pairing methods use this resolver. Ordinary DNS names and IP addresses continue to use the normal system resolver. No fixed RetroPie address is written to `/etc/hosts`. Generic container tools such as `getent` may still lack mDNS; test through the app's Connection page.

If App Lab regenerates its Compose file during a fresh import, rerun deployment or the installation step below to restore the mount. Restarting or rebooting an existing configured container retains it.

Known limitation: UNO Q restarts after Shutdown

On the tested UNO Q, a graceful `halt` still leads to an automatic restart, including when connected directly to a Mac without the powered hub. The helper requests halt correctly, but remaining stopped is not verified. Do not use loss of the website or a fixed delay as confirmation that power can safely be removed. See the installation guide for the investigation status and Arduino guidance.